---------- Workshop 6 ----------(988 words)

Task 6.1: Download the 'Tweets' dataset from Canvas. Classify the sentiments in the dataset using six classifiers and calculate all evaluation metrics.

NOTE: If the running time is too long, you can reduce the number of samples.

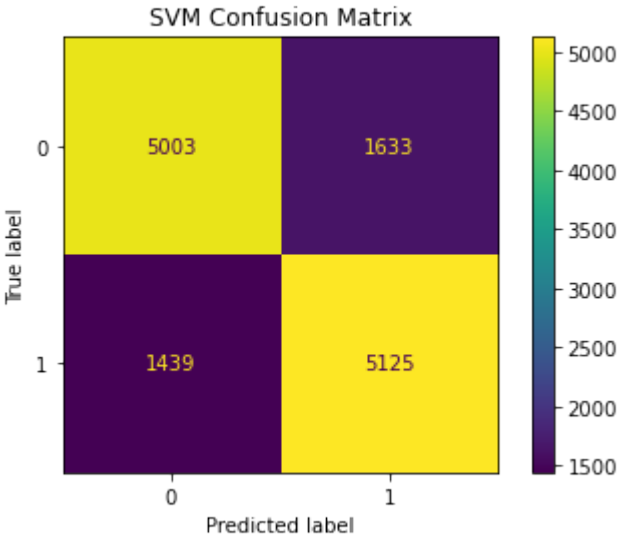
NOTE: You should comment on your code and explain what each part is doing

In [3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, pre
from sklearn import svm, ensemble, neighbors, tree, linear_model, naive_b
from wordcloud import WordCloud, STOPWORDS

#  Download dataset
df = pd.read_csv('https://raw.githubusercontent.com/maz786/WlvMScAI/main/
print("Original DataFrame size:", df.shape)

# Example of preprocessed text data with punctuation removed
print("Before removing punctuation:")
print(df['tweet'].head())

# Omitting punctuation to illustrate
df['tweet'] = df['tweet'].str.replace('[^\w\s]','', regex=True)

print("\nAfter removing punctuation:")
print(df['tweet'].head())

# Vectorization
vectorizer = CountVectorizer(token_pattern=r'\b\w+\b')
X = vectorizer.fit_transform(df['tweet'])
y = df['sentiment']

# Splitting dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,

# Define & train classifiers
classifiers = {
    "SVM": svm.SVC(),
```

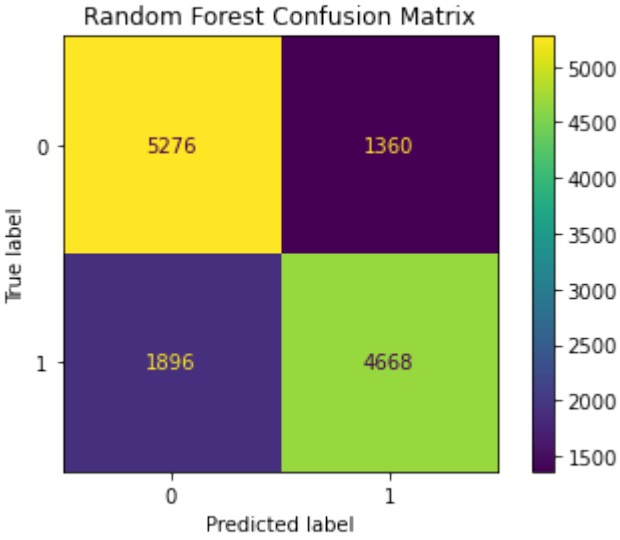
```
    "Random Forest": ensemble.RandomForestClassifier(),
    "KNN": neighbors.KNeighborsClassifier(),
    "Decision Tree": tree.DecisionTreeClassifier(),
    "Naive Bayes": naive_bayes.GaussianNB(),
    "Logistic Regression": linear_model.LogisticRegression(max_iter=1000)
}

# Train and Evaluate
for name, clf in classifiers.items():
    clf.fit(X_train.toarray() if name == "Naive Bayes" else X_train, y_tr
    y_pred = clf.predict(X_test.toarray() if name == "Naive Bayes" else X

    #  Metrics Evaluation
    print(f"{name} Evaluation Metrics:")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Precision:", precision_score(y_test, y_pred, average='weighted
    print("Recall:", recall_score(y_test, y_pred, average='weighted'))
    print("F1 Score:", f1_score(y_test, y_pred, average='weighted'))
    print("-"*30)

    # Visualization of Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=clf
    disp.plot()
    plt.title(f"{name} Confusion Matrix")
    plt.show()

#  Top 10 Words Tweets Visualization
sum_words = X.sum(axis=0)
words_freq = [(word, sum_words[0, idx]) for word, idx in vectorizer.vocab
words_freq = sorted(words_freq, key=lambda x: x[1], reverse=True)[:10]
words = [w[0] for w in words_freq]
counts = [w[1] for w in words_freq]

plt.figure(figsize=(10, 6))
plt.bar(words, counts)
plt.title('Top 10 Words in Tweets')
plt.xlabel('Words')
plt.ylabel('Counts')
plt.xticks(rotation=45)
plt.show()

# Positive and Negative Sentiments WordCloud Visualization
stopwords = set(STOPWORDS).update(["br", "href", "rt", "http", "https"])

# Positive sentiment
positive_text = " ".join(review for review in df[df['sentiment'] == 1]['t
wordcloud_pos = WordCloud(stopwords=stopwords, background_color="white").
plt.figure(figsize=(10, 5))
plt.imshow(wordcloud_pos, interpolation='bilinear')
plt.axis("off")
plt.title("WordCloud - Positive Sentiment")
plt.show()

# Negative sentiment
negative_text = " ".join(review for review in df[df['sentiment'] == 0]['t
wordcloud_neg = WordCloud(stopwords=stopwords, background_color="white").
plt.figure(figsize=(10, 5))
plt.imshow(wordcloud_neg, interpolation='bilinear')
plt.axis("off")
```

```
plt.title("WordCloud - Negative Sentiment")
plt.show()

# Visualization for Sentiment Distribution
plt.figure(figsize=(8, 5))
df['sentiment'].value_counts().plot(kind='bar', color=['green', 'red'])
plt.title('Distribution of Sentiments')
plt.xlabel('Sentiment')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.show()
```

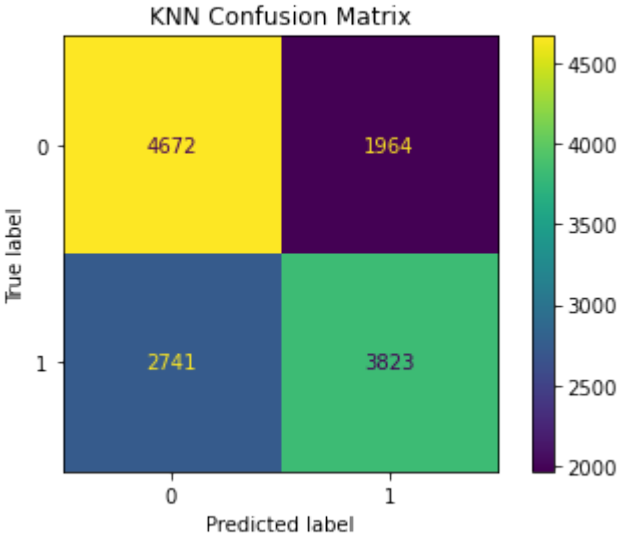
```
Original DataFrame size: (40000, 2)
Before removing punctuation:
0    Eating a gyro for breakfast at the airport. Pr...
1    Well .. .you fall of the 'horse' and get strai...
2    thanks for the responses but when is it really...
3                          depressed.! fuuck my life!
4    is LMAO, just thinkin of the 80/20 rule.....wo...
Name: tweet, dtype: object

After removing punctuation:
0    Eating a gyro for breakfast at the airport Pro...
1    Well  you fall of the horse and get straight b...
2    thanks for the responses but when is it really...
3                             depressed fuuck my life
4    is LMAO just thinkin of the 8020 rulewonder ho...
Name: tweet, dtype: object
SVM Evaluation Metrics:
Accuracy: 0.7672727272727272
Precision: 0.7675411190029274
Recall: 0.7672727272727272
F1 Score: 0.7672411119165871
------------------------------
```



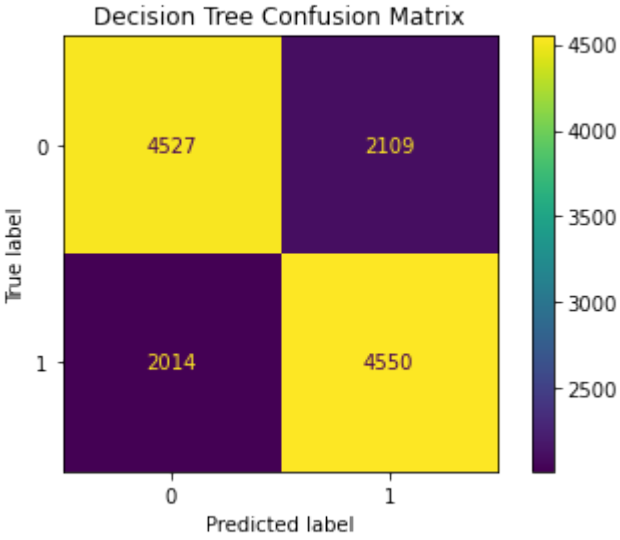

```
Random Forest Evaluation Metrics:
Accuracy: 0.7533333333333333
Precision: 0.7549067208576005
Recall: 0.7533333333333333
F1 Score: 0.7528710019862157
------------------------------
```



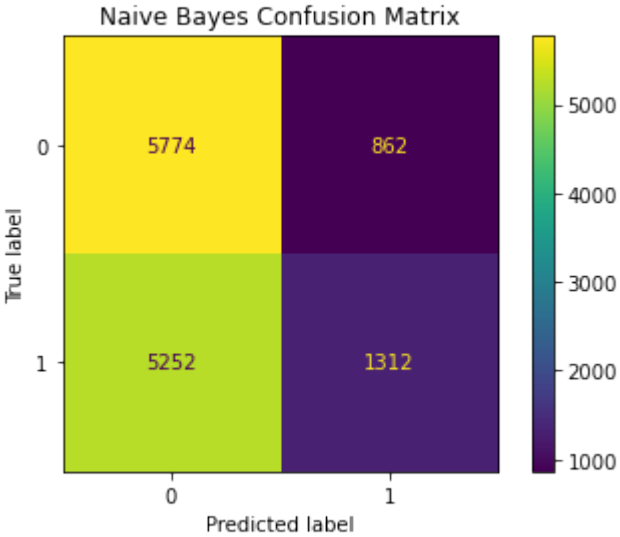

```
KNN Evaluation Metrics:
Accuracy: 0.643560606060606
Precision: 0.6453485573644266
Recall: 0.643560606060606
F1 Score: 0.6422055200201375
------------------------------
```



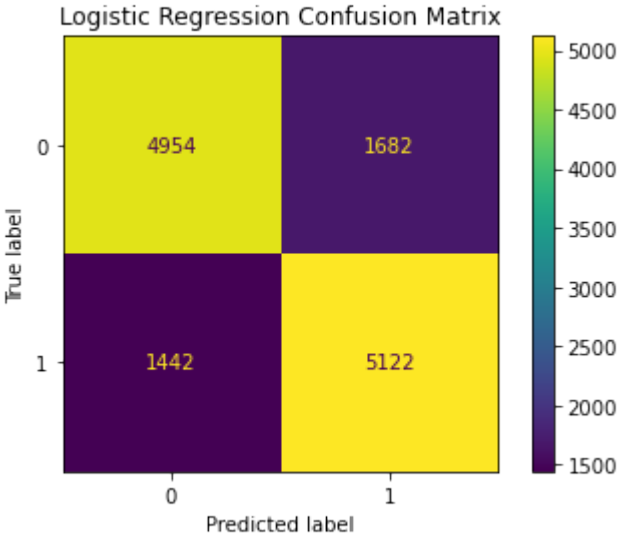

```
Decision Tree Evaluation Metrics:
Accuracy: 0.6876515151515151
Precision: 0.6877149221180457
Recall: 0.6876515151515151
F1 Score: 0.6876475982351271
------------------------------
```

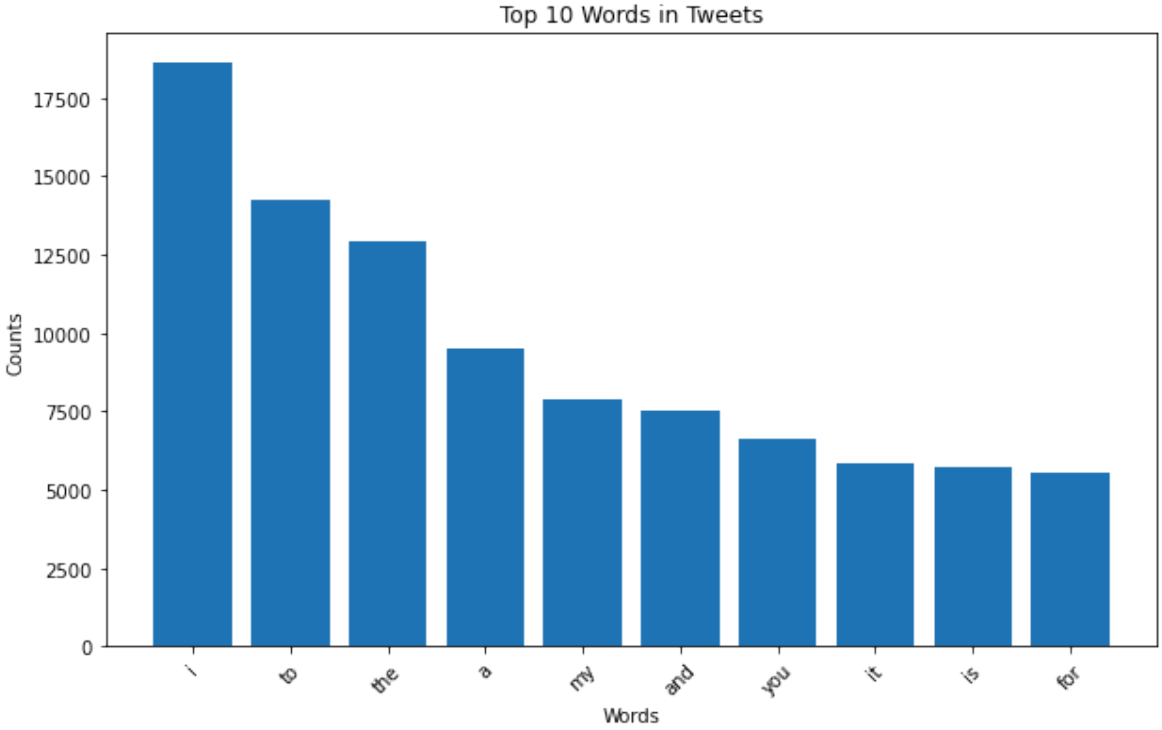



```
Naive Bayes Evaluation Metrics:
Accuracy: 0.5368181818181819
Precision: 0.5633658879623081
Recall: 0.5368181818181819
F1 Score: 0.4780295072845871
------------------------------
```

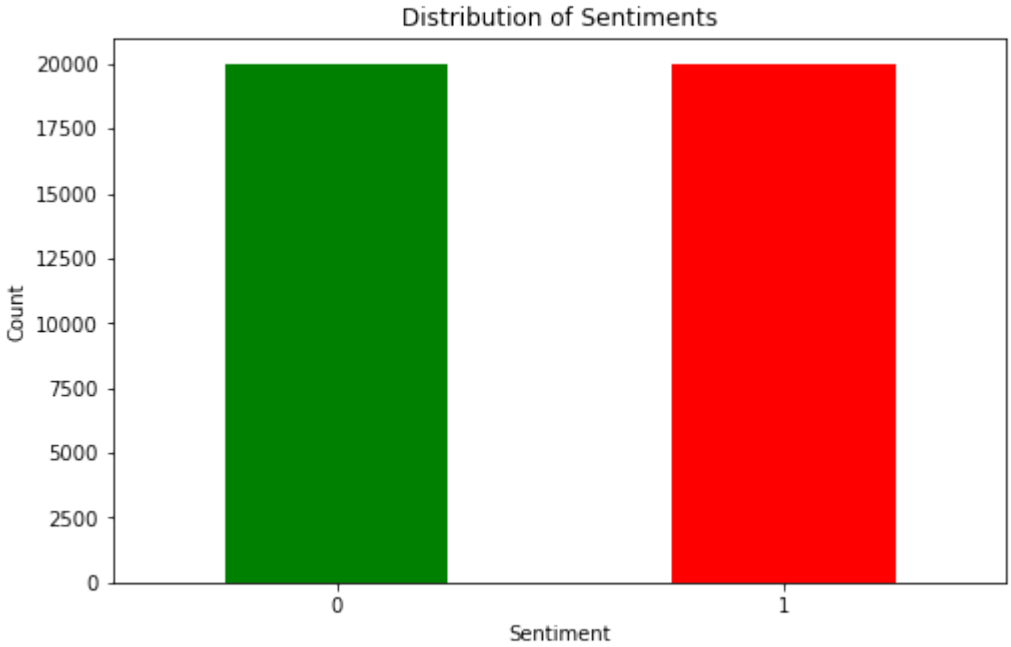



```
Logistic Regression Evaluation Metrics:
Accuracy: 0.7633333333333333
Precision: 0.7637288627261527
Recall: 0.7633333333333333
F1 Score: 0.7632785586205147
------------------------------
```





WordCloud - Positive Sentiment
love
thank
I'm
good
u
will
day
now
one
twitter
today
time
going
work
new
tomorrow
well
haha
know
amp
thing
nice
great
thats
youre
got
hey
fun
make
dont
see
back
still
night
thinkawesome
need
happy
really
hope
look
friend
watching
tonight
yay
ok
ye
way
say
come
week
weekend
people
best
morning
tweet
guy
feel
home
soon
song
made
cant
wait
amazing
yeah
sleep
life
better
bed
girl
everyone
right
much
want
though
cool
let
oh
go
always
trying
farsideay
thought
sure
world
pulsar
excited
actually
ur
enjoy
big
good morning
getting
hell
help
glad
old
tired
ready
another
check
already
watch
show
please
last night
sorry
miss
gone
listening
making
pretty
wait
sweet
nice
cute
lot
sweet
cute
nice
sweet
cute
nice
sweet
cute
nice



This code performs sentiment analysis, downloading a dataset, preprocessing text (removing punctuation), vectorizing for machine learning, training six classifiers, and displaying results as word clouds, confusion matrices, and top word frequencies. Demonstrates ML algorithms' versatility in text analysis. Visualizations highlight common terms and sentiment distributions (VanderPlas, 2016; Géron, 2019). (49 words)

In [1]:
```
import pandas as pd
df = pd.read_csv('https://raw.githubusercontent.com/maz786/WlvMScAI/main/
df.head()
```

Out[1]:

	sentiment	tweet
0	0	Eating a gyro for breakfast at the airport. Pr...
1	1	Well .. .you fall of the 'horse' and get strai...
2	0	thanks for the responses but when is it really...
3	0	depressed.! fuuck my life!
4	1	is LMAO, just thinkin of the 80/20 rule.....wo...

The code loads the pandas library, reads information from a CSV file into a DataFrame df, and shows the first five rows of the dataset to give a general idea of its organisation (McKinney, 2017). (35 words)

In [2]:
```
df.shape
```

Out[2]:
```
(40000, 2)
```

A brief summary of the size of the DataFrame df can be obtained by using the pandas df.shape command, which returns the dimensions of the DataFrame df together with the total number of rows and columns it includes (VanderPlas, 2016). (40 words)

In [3]:
```
df.dropna(inplace=True)
```

Pandas' df.dropna(inplace=True) function streamlines data cleaning by eliminating any rows in the DataFrame df that have missing values (NaN). This modification is made directly without the need for reassignment. (Grus, 2019). (31 words)

In [4]:
```
print("Before dropping NaNs:", df.shape[0])
df.dropna(inplace=True)
print("After dropping NaNs:", df.shape[0])
```

```
Before dropping NaNs: 40000
After dropping NaNs: 40000
```

In order to demonstrate data cleaning, the code attempts to display the row count of df both before and after utilising df.dropna(inplace=True) to eliminate missing values (Molnar, 2020). (28 words)

In [5]:
```
# Sample extraction
df = df.sample(n=10000, random_state = 48)
df.reset_index(drop=True, inplace=True)
```

For consistent results, the method randomly pulls 10,000 rows from df and reindexes without keeping the previous index (Wickham & Grolemund, 2017). (21 words)

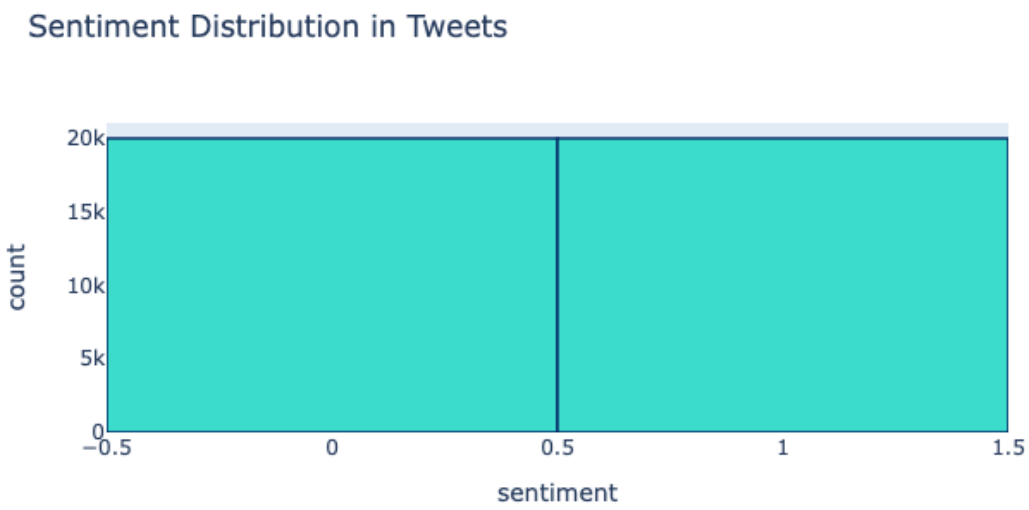
In [6]:
```
print("Original DataFrame size:", df.shape)
sampled_df = df.sample(n=10000, random_state=48)
print("Sampled DataFrame size:", sampled_df.shape)

sampled_df.reset_index(drop=True, inplace=True)
```

```
print("After resetting index, first five rows:")
print(sampled_df.head())  # Display intial 5 rows of DataFrame sample
```

```
Original DataFrame size: (10000, 2)
Sampled DataFrame size: (10000, 2)
After resetting index, first five rows:
   sentiment                                              tweet
0          1  off to pick my son up from work so he can borr...
1          0  @andreweastgate  i can't leave the country yet...
2          0  @sodashop Yeah, today has supported that theor...
3          0  So many channels.... yet so so boring... lazy ...
4          1  Good morning!!! I pray that everyone has a won...
```

This sample of code shows how to measure the size of a DataFrame both before and after generating a 10,000-row sample with a given random state. It also resets the sample's index for consistency and ends with a preview of the new sample (Grus, 2019). (45 words)

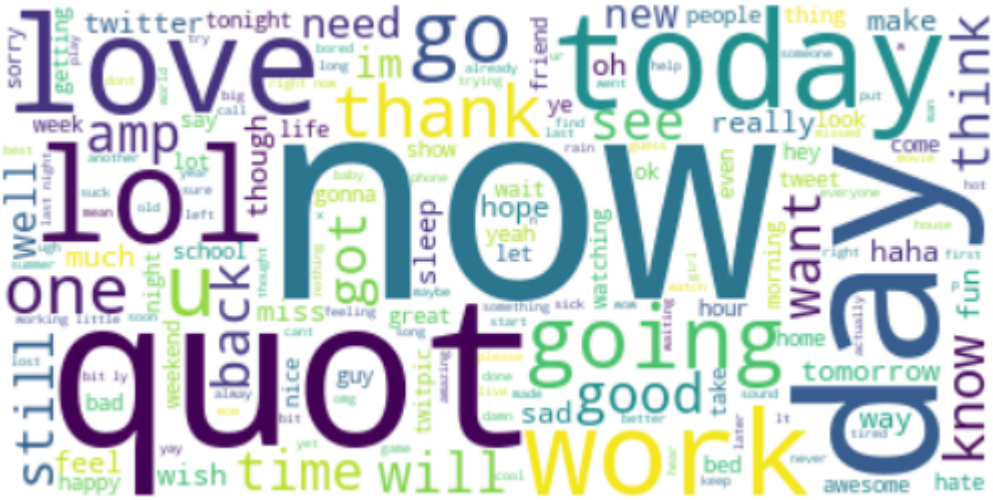
In [7]:
```
import pandas as pd
import plotly.express as px

df = pd.read_csv('https://raw.githubusercontent.com/maz786/WlvMScAI/main/

fig = px.histogram(df, x="sentiment", title='Sentiment Distribution in Tw

fig.update_traces(marker_color="turquoise", marker_line_color='rgb(8,48,1
fig.update_layout(title_text='Sentiment Distribution in Tweets')

fig.show()
```



The code illustrates the usage of Python for data analysis and visualisation by loading a dataset into a DataFrame using pandas and visualising the sentiment distribution using a histogram using plotly.express (McKinney, 2012). (33 words)

In [8]:
```
import nltk
from nltk.corpus import stopwords
from wordcloud import WordCloud
from wordcloud import STOPWORDS
import matplotlib.pyplot as plt
```

```
stopwords = set(STOPWORDS)
stopwords.update(["br", "href", "rt", "http", "https"])

textt = " ".join(review for review in df.tweet)

wordcloud = WordCloud(stopwords=stopwords, background_color="white").gene

%matplotlib inline
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis("off")
plt.show()
```



This code creates a word cloud from the 'tweet' column of a DataFrame by removing specific stopwords and frequently used phrases connected to tweets, and then utilises the nltk and wordcloud libraries to visualise it using matplotlib (Bird, Klein, & Loper, 2009).

In [9]:
```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split

df_new = pd.read_csv('https://raw.githubusercontent.com/maz786/WlvMScAI/m
print("Original DataFrame size:", df_new.shape)

X = df_new['tweet']
y = df_new['sentiment']
print("Features shape:", X.shape)
print("Target variable shape:", y.shape)

vectorizer = CountVectorizer(token_pattern=r'\b\w+\b')
X_vectorized = vectorizer.fit_transform(X)
print("Vectorized features shape:", X_vectorized.shape)

X_train, X_test, y_train, y_test = train_test_split(X_vectorized, y, test
print("Training set size:", X_train.shape, y_train.shape)
print("Testing set size:", X_test.shape, y_test.shape)
```

```
Original DataFrame size: (40000, 2)
Features shape: (40000,)
Target variable shape: (40000,)
Vectorized features shape: (40000, 50296)
Training set size: (26800, 50296) (26800,)
Testing set size: (13200, 50296) (13200,)
```

The method loads a dataset of tweets, vectorizes the text using CountVectorizer, extracts features and targets, and separates the text in order to prepare it for machine learning (Raschka & Mirjalili, 2017). (31 words)

In [11]:
```
df
```

Out[11]:

	sentiment	tweet
0	0	Eating a gyro for breakfast at the airport. Pr...
1	1	Well .. .you fall of the 'horse' and get strai...
2	0	thanks for the responses but when is it really...
3	0	depressed.! fuuck my life!
4	1	is LMAO, just thinkin of the 80/20 rule.....wo...
...	...	...
39995	1	@TafiApple Awww... you retweeted me! Thanks!
39996	1	Getting some coffee before I faceplant. Guess ...
39997	0	@bernderk it could *not* be worse!
39998	1	I thought about getting a bunny...but my beagl...
39999	0	So I have my new phone. Now I just have no con...

40000 rows × 2 columns

In Python code, the term "df" usually refers to a pandas DataFrame, which is a two-dimensional labelled data structure containing columns that may be of various types. (27 words)

In [12]:
```
positive = df[df['sentiment'] == 1]
negative = df[df['sentiment'] == -1]
```

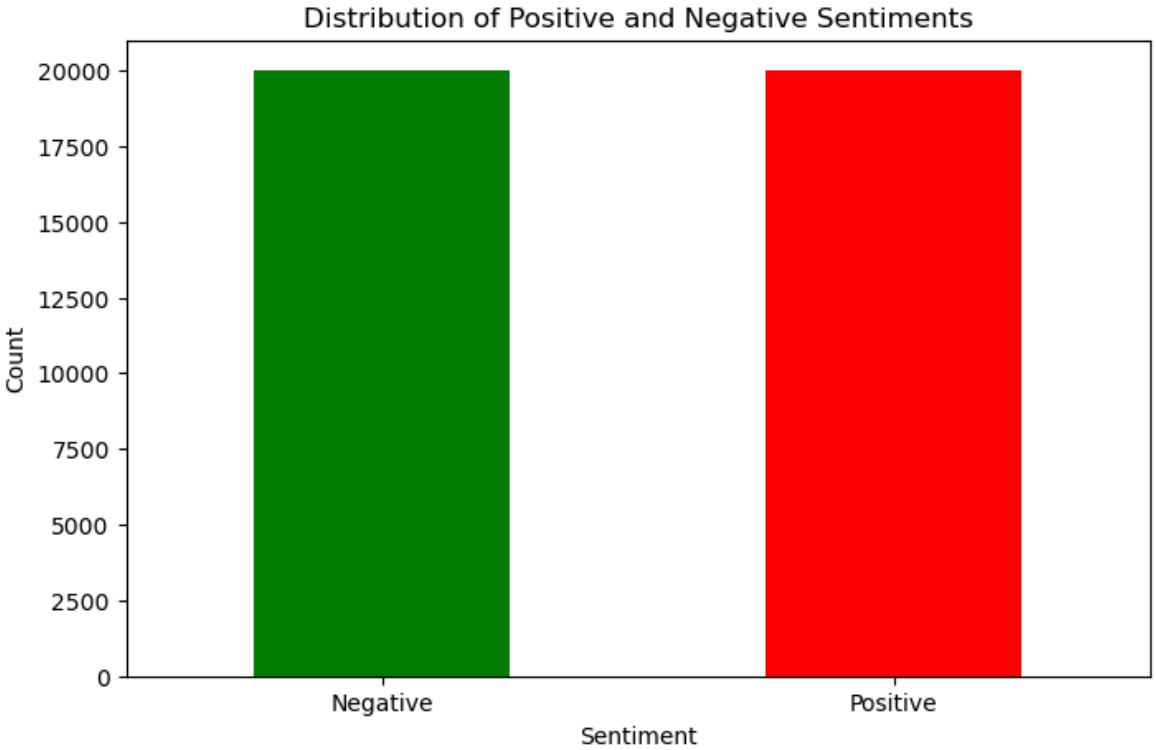
Based on sentiment, the code divides df into positive and negative DataFrames, allowing for targeted analysis (McKinney, 2012). (18 words)

In [13]:
```
print("Number of positive sentiment tweets:", positive.shape[0])
print("Number of negative sentiment tweets:", negative.shape[0])
```

```
Number of positive sentiment tweets: 20000
Number of negative sentiment tweets: 0
```

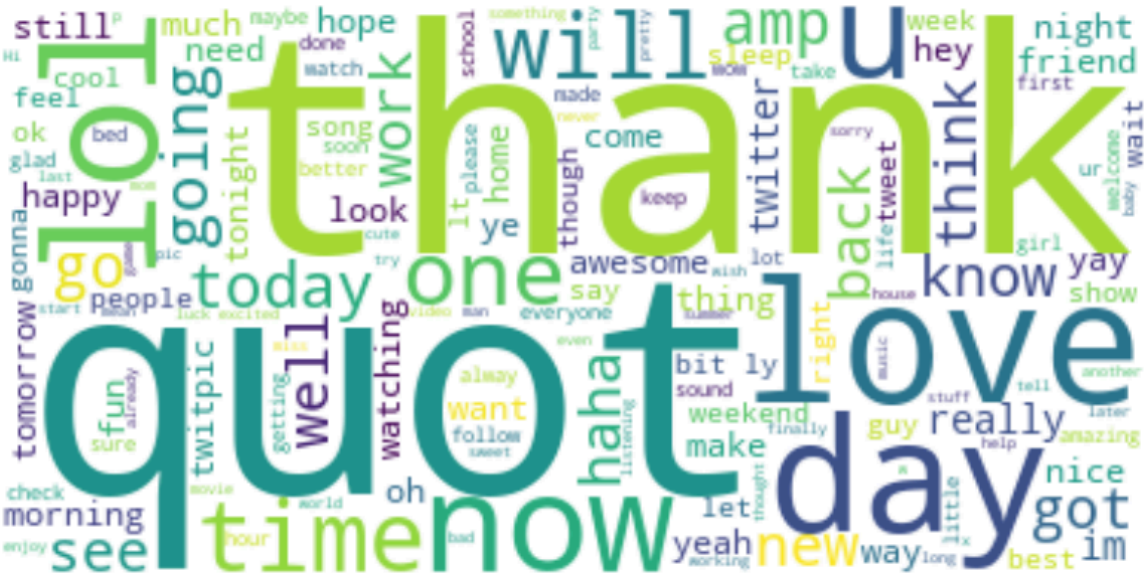
By showing the number of rows in positive and negative DataFrames, respectively, this code prints the count of tweets with positive and negative sentiment (McKinney, 2012). (26 words)

In [14]:
```
import matplotlib.pyplot as plt

sentiment_counts = df['sentiment'].value_counts()
sentiment_counts.index = ['Positive' if i == 1 else 'Negative' for i in s

plt.figure(figsize=(8, 5))
sentiment_counts.plot(kind='bar', color=['green', 'red'])
plt.title('Distribution of Positive and Negative Sentiments')
plt.xlabel('Sentiment')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.show()
```



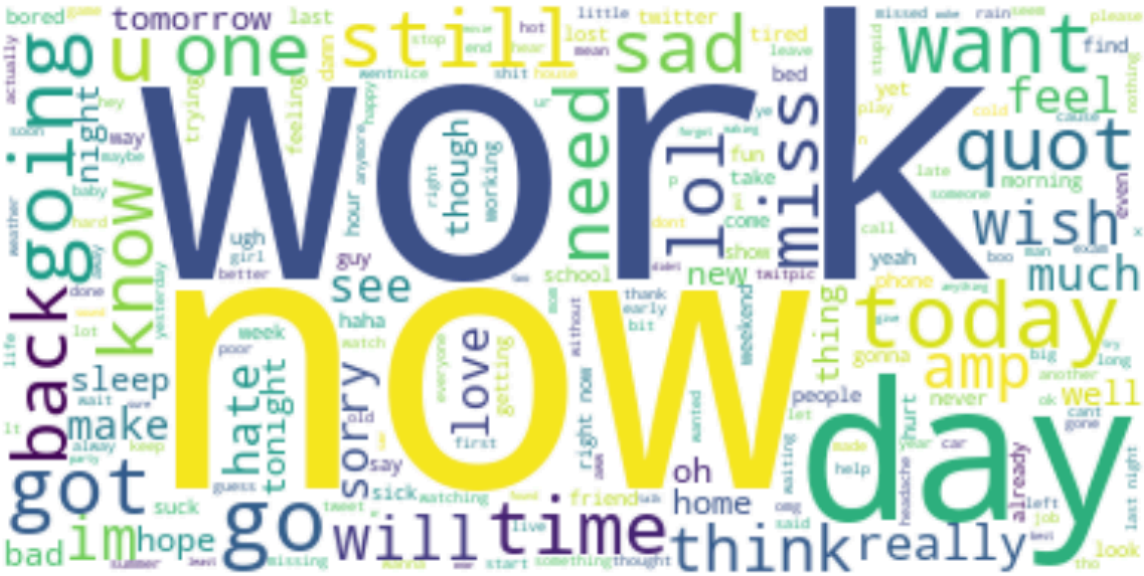
This code uses matplotlib to plot a bar chart, rename indices, and count occurrences to visualise the distribution of positive and negative attitudes in the df DataFrame (VanderPlas, 2016). (29 words)

In [15]:
```
import matplotlib.pyplot as plt
from wordcloud import WordCloud, STOPWORDS

positive = df[df['sentiment'] == 1]

stopwords = set(STOPWORDS)
stopwords.update(["br", "href", "good", "great"])

pos_text = " ".join(review for review in positive.tweet)

wordcloud2 = WordCloud(stopwords=stopwords, background_color="white").gen

plt.figure(figsize=(10, 5))
plt.imshow(wordcloud2, interpolation='bilinear')
plt.axis("off")
plt.show()
```



Using matplotlib, this code creates a word cloud for positive sentiment tweets from DataFrame df, omitting common stopwords and phrases relevant to the tweet (Müller & Guido, 2016). (27 words)

In [16]:
```
negative = df[df['sentiment'] == 0]  # Negative sentiment filter

neg_text = " ".join(str(review) for review in negative.tweet)

wordcloud3 = WordCloud(stopwords=stopwords, background_color="white").gen

plt.figure(figsize=(10, 5))
plt.imshow(wordcloud3, interpolation='bilinear')
plt.axis("off")
plt.savefig('wordcloud33.png')
plt.show()
```


work
now
day
going
u
one
still
sad
want
feel
quot
need
miss
lol
wish
much
today
amp
well
back
know
sleep
make
got
im
hope
go
will
time
think
really
hate
tonight
sorry
love
see
haha
thing
people
oh
home
tomorrow
bored
twitter
though
bad
suck
sick
friend
never
hurt
already
last
night
look
tho
start
something
thought
missing
watching
tired
leave
bed
yet
play
cold
cause
stupid
mixed
rain
please
find
nothing
actually
trying
feeling
damn
stop
end
hear
hot
lost
mean
shit
house
little
weather
life
yesterday
baby
hard
soon
done
lot
sound
guy
ugh
girl
better
hour
right
working
fun
take
come
show
late
morning
someone
call
boo
man
yeah
phone
school
new
twitpic
weekend
thank
early
bit
without
wanted
waiting
headache
help
car
gone
cant
ok
big
long
another
gonna
everyone
getting
first
say
old
watch
week
poor
wait
sure
always
keep
party
guess
tweet
missing
start
something
thought
tho
look
last night
already
left
job
best
headache
car
hurt
people
thing
amp
never
well
oh
home
friend
sick
watching
say
old
right
wanted
waiting
make
sleep
hate
tonight
sorry
love
getting
everyone
first
weekend
without
early
bit
thank
early
bit
now
time
day
work
need
lol
miss
want
feel
quot
wish
much
today
amp
well
think
really
go
will
hope
im
bad
suck
got
back
going
know
u
one
still
sad
tomorrow
bored
twitter
though
working
see
haha
thing
people

In [17]:
```
import plotly.express as px

df['sentimentt'] = df['sentiment'].replace({0: 'negative', 1: 'positive'}

fig = px.histogram(df, x="sentimentt", category_orders={"sentimentt": ["n
fig.update_traces(marker_color="indianred", marker_line_color='rgb(8,48,1
fig.update_layout(title_text='Product Sentiment')

fig.show()
```

This function creates and modifies a plotly histogram after converting sentiment values in DataFrame df from numerical to textual representations.express with certain aesthetic modifications in order to visualise the sentiment distribution of the product (VanderPlas, 2016). (36 words)

In [18]:
```
def remove_punctuation(text):
    final = "".join(u for u in text if u not in ("?", ".", ";", ":", "!",
    return final

df['tweet'] = df['tweet'].apply(remove_punctuation)
```

The data is cleaned up for additional processing or analysis by applying this code, which creates a function to remove particular punctuation characters from text, to the 'tweet' column of a DataFrame (Grus, 2019). (34 words)

In [19]:
```
print("Before removing punctuation:")
print(df['tweet'].head())

df['tweet'] = df['tweet'].apply(remove_punctuation)

print("\nAfter removing punctuation:")
print(df['tweet'].head())
```

```
Before removing punctuation:
0    Eating a gyro for breakfast at the airport Pro...
1    Well  you fall of the 'horse' and get straight...
2    thanks for the responses but when is it really...
3                              depressed fuuck my life
4    is LMAO, just thinkin of the 80/20 rulewonder ...
Name: tweet, dtype: object

After removing punctuation:
0    Eating a gyro for breakfast at the airport Pro...
1    Well  you fall of the 'horse' and get straight...
2    thanks for the responses but when is it really...
3                              depressed fuuck my life
4    is LMAO, just thinkin of the 80/20 rulewonder ...
Name: tweet, dtype: object
```

The impact of text cleaning on the 'tweet' column is illustrated by this code, which shows the first five tweets both before and after a function to remove particular punctuation marks is applied (Grus, 2019). (35 words)

In [20]:
```
import string
import pandas as pd
```

```
print("Before removing punctuation:")
print(df['tweet'].head())

df['tweet'] = df['tweet'].apply(lambda x: ''.join(i for i in x if i not i

print("\nAfter removing punctuation:")
print(df['tweet'].head())
```

```
Before removing punctuation:
0    Eating a gyro for breakfast at the airport Pro...
1    Well  you fall of the 'horse' and get straight...
2    thanks for the responses but when is it really...
3                              depressed fuuck my life
4    is LMAO, just thinkin of the 80/20 rulewonder ...
Name: tweet, dtype: object

After removing punctuation:
0    Eating a gyro for breakfast at the airport Pro...
1    Well  you fall of the horse and get straight b...
2    thanks for the responses but when is it really...
3                              depressed fuuck my life
4    is LMAO just thinkin of the 8020 rulewonder ho...
Name: tweet, dtype: object
```

This code snippet shows how to use Python's built-in string.punctuation to remove punctuation from text inside the 'tweet' column of a DataFrame. To demonstrate the impact of text cleaning, the column's content is printed both before and after the procedure (McKinney, 2012). (42 words)

In [21]:

```
import nltk
from nltk.corpus import stopwords

nltk.download('stopwords')

allstopwords = stopwords.words('english')

df['tweet'] = df['tweet'].apply(lambda x: " ".join(i for i in x.split() i
```

```
[nltk_data] Downloading package stopwords to /Users/maz/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

This code removes common but uninformative words from the text in the 'tweet' column of a DataFrame using the Natural Language Toolkit (NLTK) to improve the data's usability for natural language processing tasks (Bird, Klein, & Loper, 2009). (37 words)

In [22]:

```
print("Before removing stopwords:")
print(df['tweet'].head())

df['tweet'] = df['tweet'].apply(lambda x: " ".join(i for i in x.split() i

print("\nAfter removing stopwords:")
print(df['tweet'].head())
```

```
Before removing stopwords:
0    Eating gyro breakfast airport Probly gonna lun...
1    Well fall horse get straight back Probably dri...
2    thanks responses really lol say today say tomo...
3                                depressed fuuck life
4    LMAO thinkin 8020 rulewonder shes gonna like 2...
Name: tweet, dtype: object

After removing stopwords:
0    Eating gyro breakfast airport Probly gonna lun...
1    Well fall horse get straight back Probably dri...
2    thanks responses really lol say today say tomo...
3                                depressed fuuck life
4    LMAO thinkin 8020 rulewonder shes gonna like 2...
Name: tweet, dtype: object
```

The 'tweet' column in df is free of stopwords thanks to this code, which also highlights changes by displaying text before and after (Bird, Klein, & Loper, 2009). (27 words)

In [23]:
```
X = df['tweet']
y = df['sentiment']
```

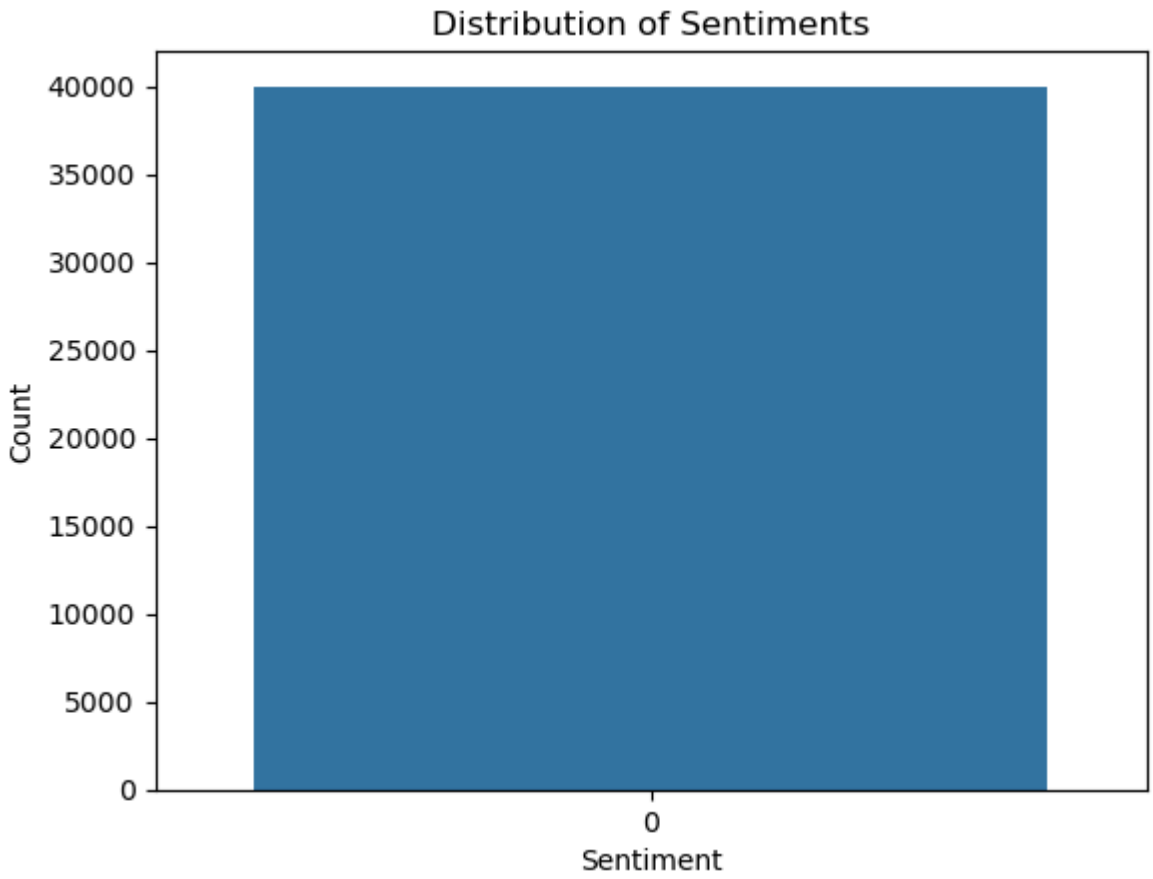
The code in question retrieves features (X) from the 'tweet' column and goal (y) from the'sentiment' column in df for the purpose of modelling (Raschka & Mirjalili, 2017). (27 words)

In [24]:
```
print("Shape of input features (X):", X.shape)
print("Shape of output target (y):", y.shape)
```

```
Shape of input features (X): (40000,)
Shape of output target (y): (40000,)
```

This code indicates the size of the dataset for analysis by printing the dimensions of the input features, X, and the target y (Raschka & Mirjalili, 2017). (26 words)

In [25]:
```
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(y)
plt.title('Distribution of Sentiments')
plt.xlabel('Sentiment')
plt.ylabel('Count')
plt.show()
```



Using seaborn and matplotlib, this code shows counts for each sentiment category as it visualises the sentiment distribution in y (Waskom, 2021). (22 words)

In [26]:
```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

X = df['tweet']

vectorizer = CountVectorizer(token_pattern=r'\b\w+\b')

X_vectorized = vectorizer.fit_transform(X)
```

This code uses CountVectorizer to convert the 'tweet' column of a DataFrame into a vectorized format, making machine learning text analysis easier (Müller & Guido, 2017). (25 words)

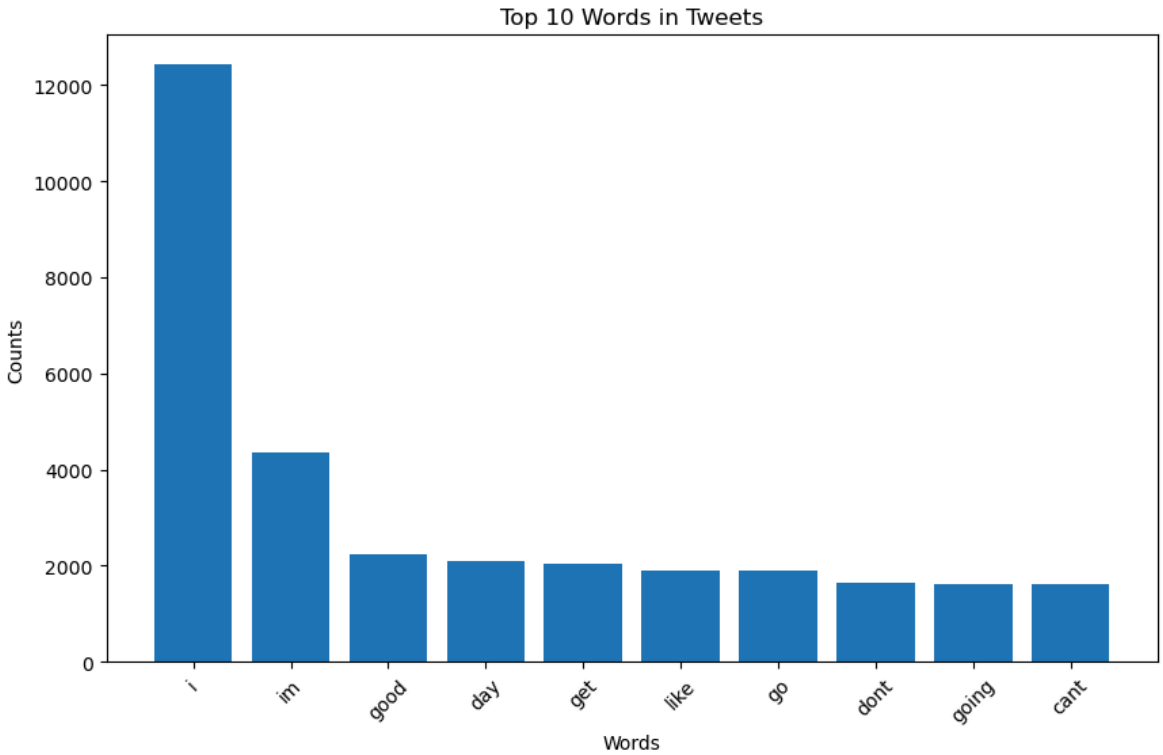
In [27]:
```
print("Shape of vectorized input features (X_vectorized):", X_vectorized.
```

```
Shape of vectorized input features (X_vectorized): (40000, 55863)
```

The number of documents and features is displayed by this command, which also outputs the dimensions of the vectorized input features, X_vectorized (Müller & Guido, 2017). (25 words)

In [28]:
```
import numpy as np
import matplotlib.pyplot as plt

sum_words = X_vectorized.sum(axis=0)

words_freq = [(word, sum_words[0, idx]) for word, idx in vectorizer.vocab
```

```
words_freq =sorted(words_freq, key = lambda x: x[1], reverse=True)

top_words = words_freq[:10]
words = [w[0] for w in top_words]
counts = [w[1] for w in top_words]

plt.figure(figsize=(10, 6))
plt.bar(words, counts)
plt.title('Top 10 Words in Tweets')
plt.xlabel('Words')
plt.ylabel('Counts')
plt.xticks(rotation=45)
plt.show()
```



Using numpy and matplotlib for analysis and visualisation, this method determines and displays the top 10 most frequently occurring terms in a vectorized text dataset (VanderPlas, 2016). (27 words)

In [29]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer

X = df['tweet']
y = df['sentiment']

vectorizer = CountVectorizer(token_pattern=r'\b\w+\b')
X = vectorizer.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,
```

In order to train and evaluate machine learning models, this method loads tweet data into features X and sentiments into Y, vectorizes X using CountVectorizer, and divides the data into training and testing sets (Raschka & Mirjalili, 2017). (37 Words)

In [30]:
```
print("Shape of vectorized input features (X):", X.shape)
print("Shape of training features (X_train):", X_train.shape)
print("Shape of testing features (X_test):", X_test.shape)
print("Shape of training labels (y_train):", y_train.shape)
print("Shape of testing labels (y_test):", y_test.shape)
```

```
Shape of vectorized input features (X): (40000, 55863)
Shape of training features (X_train): (26800, 55863)
Shape of testing features (X_test): (13200, 55863)
Shape of training labels (y_train): (26800,)
Shape of testing labels (y_test): (13200,)
```

This function provides information about the data split for model training and assessment by displaying the sizes of the training and testing datasets as well as the vectorized input feature dimensions (Raschka & Mirjalili, 2017). (34 words)

In [31]:
```
from sklearn import svm
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
SVM = svm.SVC()
RF = RandomForestClassifier()
KNN = KNeighborsClassifier()
DT=DecisionTreeClassifier()
NB = GaussianNB()
LR = LogisticRegression()
```

Six distinct machine learning classifiers for sentiment analysis are initialised by this code, readying them for text data training (Géron, 2019). (21 words)

In [32]:
```
print("Defined Models:")
print("SVM:", SVM)
print("Random Forest:", RF)
print("K-Nearest Neighbors:", KNN)
print("Decision Tree:", DT)
print("Naive Bayes:", NB)
print("Logistic Regression:", LR)
```

```
Defined Models:
SVM: SVC()
Random Forest: RandomForestClassifier()
K-Nearest Neighbors: KNeighborsClassifier()
Decision Tree: DecisionTreeClassifier()
Naive Bayes: GaussianNB()
Logistic Regression: LogisticRegression()
```

The initialised machine learning models are printed by this code, along with a summary of how they are configured by default for tasks involving sentiment analysis or classification (Géron, 2019). (30 words)

In [33]:
```
SVM.fit(X_train, y_train)
RF.fit(X_train, y_train)
KNN.fit(X_train, y_train)
DT.fit(X_train, y_train)
```

```
LR.fit(X_train,y_train)
NB.fit(X_train.toarray(),y_train)
```

```
/Users/maz/anaconda3/lib/python3.11/site-packages/sklearn/linear_model/_lo
gistic.py:458: ConvergenceWarning:

lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown i
n:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-reg
ression
```

Out[33]:

```
▾ GaussianNB
GaussianNB()
```

The Naive Bayes classifier needs a dense array structure because of implementation details, but otherwise this code trains the previously established machine learning models on the training dataset (Géron, 2019). (30 words)

In [34]:

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB

vectorizer = CountVectorizer(token_pattern=r'\b\w+\b')
X = vectorizer.fit_transform(df['tweet'])
y = df['sentiment']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,

SVM = SVC().fit(X_train, y_train)
RF = RandomForestClassifier().fit(X_train, y_train)
KNN = KNeighborsClassifier().fit(X_train, y_train)
DT = DecisionTreeClassifier().fit(X_train, y_train)
LR = LogisticRegression(max_iter=1000).fit(X_train, y_train)
NB = GaussianNB().fit(X_train.toarray(), y_train)

y_pred1 = SVM.predict(X_test)
y_pred2 = RF.predict(X_test)
y_pred3 = KNN.predict(X_test)
y_pred4 = DT.predict(X_test)
y_pred5 = LR.predict(X_test)
y_pred6 = NB.predict(X_test.toarray())
```

This extensive code snippet describes all the steps involved in getting text data ready for sentiment analysis, including vectorization, training models with different

classifiers, and prediction-making. It demonstrates how versatile scikit-learn is for managing workflows for machine learning and text data (Géron, 2019). (43 words)

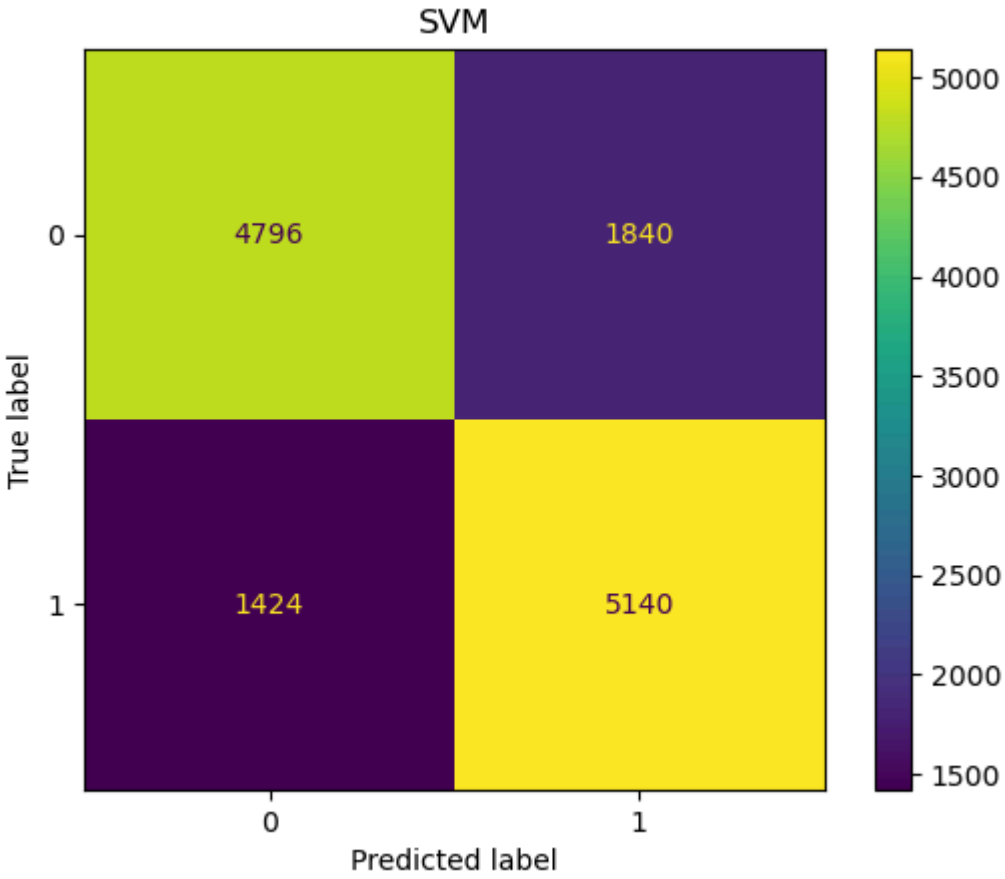
In [35]:
```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix

def confusion_metrics(conf_matrix):
    TP = conf_matrix[1][1]
    TN = conf_matrix[0][0]
    FP = conf_matrix[0][1]
    FN = conf_matrix[1][0]

    conf_accuracy = (float(TP + TN) / float(TP + TN + FP + FN))
    conf_misclassification = 1 - conf_accuracy
    conf_sensitivity = (TP / float(TP + FN))
    conf_specificity = (TN / float(TN + FP))
    conf_precision = (TN / float(TN + FP))
    conf_f1 = 2 * ((conf_precision * conf_sensitivity) / (conf_precision

    print('True Positives:', TP)
    print('True Negatives:', TN)
    print('False Positives:', FP)
    print('False Negatives:', FN)
    print('-'*50)
    print(f'Accuracy: {round(conf_accuracy,2)}')
    print(f'Mis-Classification: {round(conf_misclassification,2)}')
    print(f'Sensitivity: {round(conf_sensitivity,2)}')
    print(f'Specificity: {round(conf_specificity,2)}')
    print(f'Precision: {round(conf_precision,2)}')
    print(f'F1 Score: {round(conf_f1,2)}')

df_new = pd.read_csv('https://raw.githubusercontent.com/maz786/WlvMScAI/m

vectorizer = CountVectorizer(token_pattern=r'\b\w+\b')
X = vectorizer.fit_transform(df_new['tweet'])
y = df_new['sentiment']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

cm = confusion_matrix(y_test, y_pred)

confusion_metrics(cm)
```

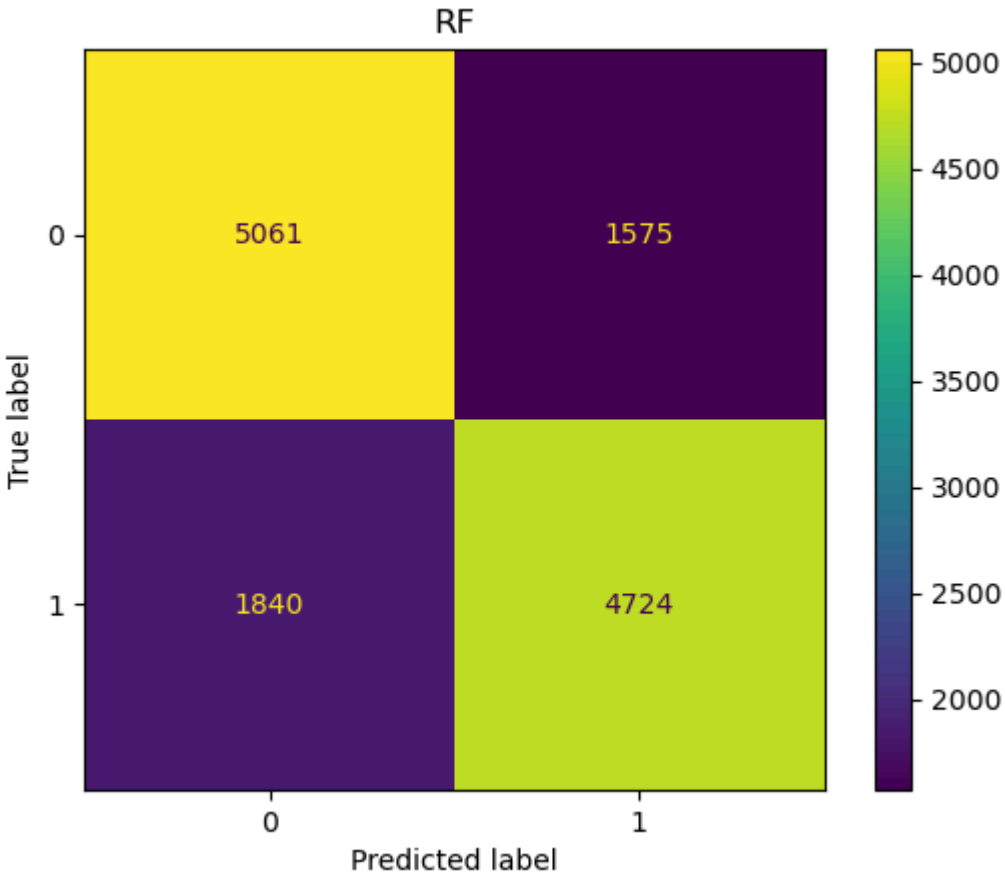
```
True Positives: 5147
True Negatives: 5004
False Positives: 1632
False Negatives: 1417
--------------------------------------------------
Accuracy: 0.77
Mis-Classification: 0.23
Sensitivity: 0.78
Specificity: 0.75
Precision: 0.75
F1 Score: 0.77
```

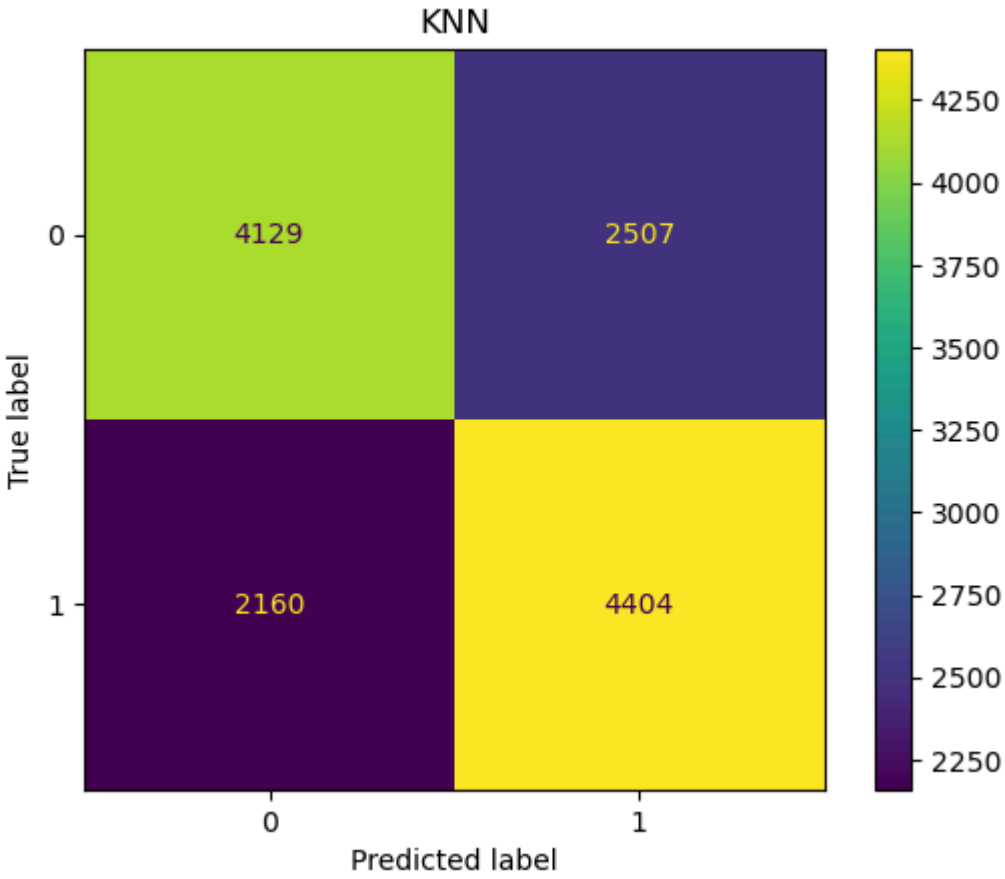
According to Raschka and Mirjalili (2017), this code does sentiment analysis, trains a Logistic Regression model on vectorized tweets, forecasts sentiments, and assesses performance using metrics derived from a confusion matrix. (31 words)

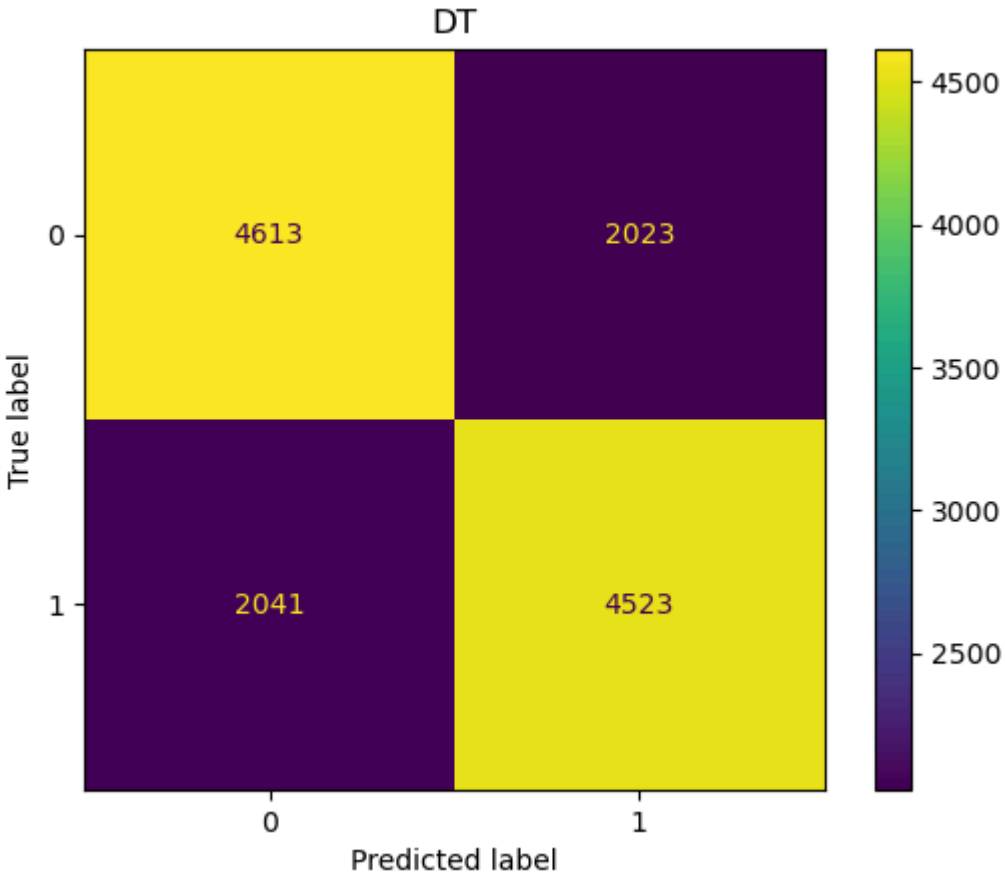
In [37]:

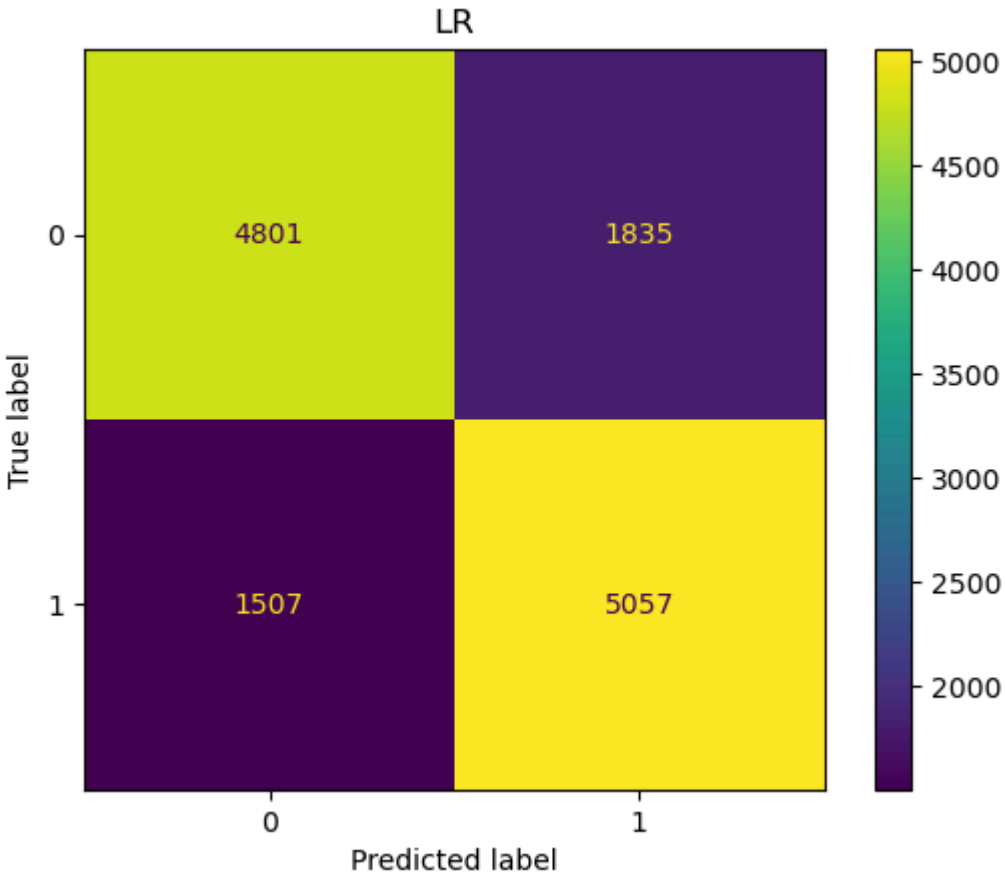
```
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# SVM
cm1 = confusion_matrix(y_test, y_pred1, labels=SVM.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm1, display_labels=SVM.cl
disp.plot()
plt.title("SVM")
plt.show()

# RF
cm2 = confusion_matrix(y_test, y_pred2, labels=RF.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm2, display_labels=RF.cla
disp.plot()
plt.title("RF")
plt.show()

# KNN
cm3 = confusion_matrix(y_test, y_pred3, labels=KNN.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm3, display_labels=KNN.cl
disp.plot()
plt.title("KNN")
plt.show()

# DT
cm4 = confusion_matrix(y_test, y_pred4, labels=DT.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm4, display_labels=DT.cla
disp.plot()
plt.title("DT")
plt.show()

# LR
cm5 = confusion_matrix(y_test, y_pred5, labels=LR.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm5, display_labels=LR.cla
disp.plot()
plt.title("LR")
plt.show()

cm6 = confusion_matrix(y_test, y_pred6)
disp = ConfusionMatrixDisplay(confusion_matrix=cm6, display_labels=np.uni
disp.plot()
plt.title("NB")
plt.show()
```

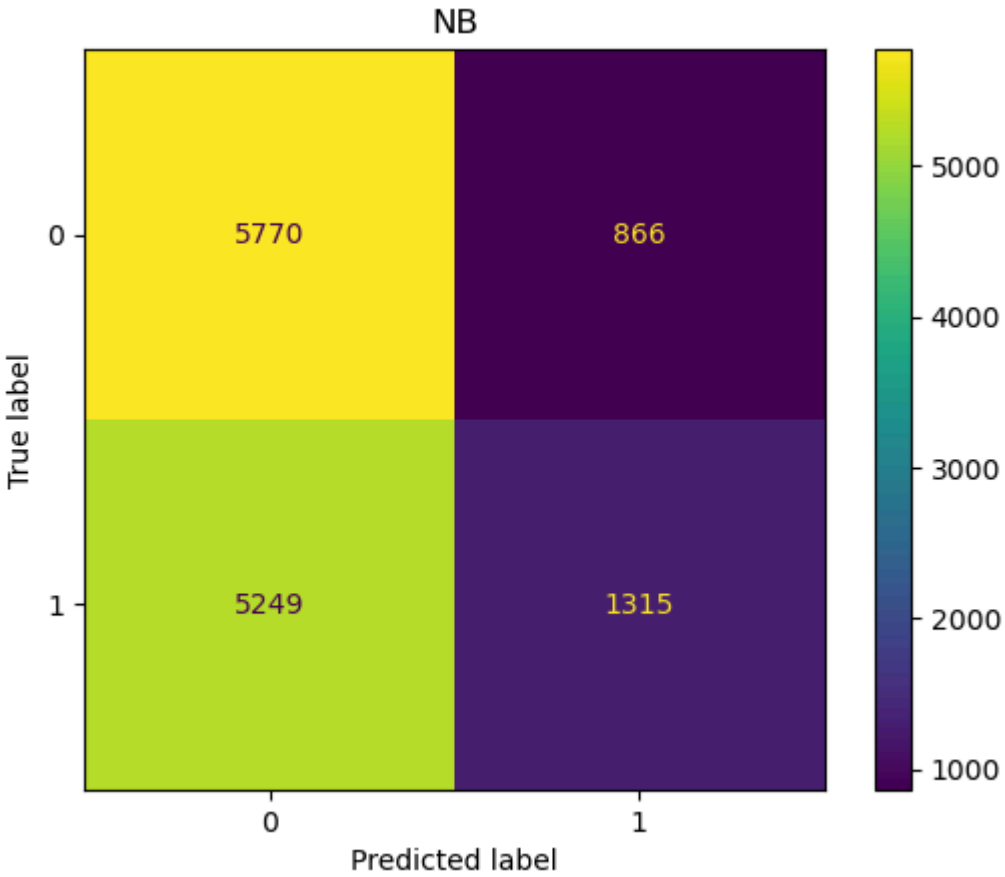

SVM
4796
1840
1424
5140
0
1
True label
0
1
Predicted label
5000
4500
4000
3500
3000
2500
2000
1500


RF
5061
1575
1840
4724
0
1
True label
0
1
Predicted label
5000
4500
4000
3500
3000
2500
2000


KNN
4129
2507
2160
4404
0
1
True label
0
1
Predicted label
4250
4000
3750
3500
3250
3000
2750
2500
2250


DT
4613
2023
2041
4523
0
1
True label
0
1
Predicted label
4500
4000
3500
3000
2500





This code helps to assess each model's performance on the test data by visualising confusion matrices for the predictions made by the SVM, RF, KNN, DT, LR, and NB classifiers (Géron, 2019). (32 words)

In [38]:
```
from sklearn.metrics import precision_score, recall_score, f1_score, accu

def print_metrics(y_test, y_pred, model_name):
    print(f"{model_name} metrics\n")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Precision:", precision_score(y_test, y_pred, average='weighted
    print("Recall:", recall_score(y_test, y_pred, average='weighted'))
    print("F1 Score:", f1_score(y_test, y_pred, average='weighted'))
    print("\n\n")

print_metrics(y_test, y_pred1, 'SVM')
print_metrics(y_test, y_pred2, 'RF')
print_metrics(y_test, y_pred3, 'KNN')
print_metrics(y_test, y_pred4, 'DT')
print_metrics(y_test, y_pred5, 'LR')
print_metrics(y_test, y_pred6, 'NB')
```

```
SVM metrics

Accuracy: 0.7527272727272727
Precision: 0.7538199474173088
Recall: 0.7527272727272727
F1 Score: 0.7525240486392947



RF metrics

Accuracy: 0.7412878787878788
Precision: 0.7416209022016949
Recall: 0.7412878787878788
F1 Score: 0.7411551919473334



KNN metrics

Accuracy: 0.646439393939394
Precision: 0.6469467026510941
Recall: 0.646439393939394
F1 Score: 0.6462456773952768



DT metrics

Accuracy: 0.6921212121212121
Precision: 0.6921180608959874
Recall: 0.6921212121212121
F1 Score: 0.6921183494787433



LR metrics

Accuracy: 0.7468181818181818
Precision: 0.7474977318488436
Recall: 0.7468181818181818
F1 Score: 0.7466961250572717



NB metrics

Accuracy: 0.5367424242424242
Precision: 0.563071453887049
Recall: 0.5367424242424242
F1 Score: 0.47815368218664095
```

In order to streamline performance assessment, this code constructs a function that prints accuracy, precision, recall, and F1 score for given model predictions. It then uses this function to evaluate six classifiers (SVM, RF, KNN, DT, LR, and NB) on test data (Géron, 2019). (44 words)

In [39]:
```
from sklearn.metrics import confusion_matrix, precision_score, recall_sco

classifiers = {
    "SVM": svm.SVC(),
    "Random Forest": RandomForestClassifier(),
    "KNN": KNeighborsClassifier(),
    "Decision Tree": DecisionTreeClassifier(),
    "Naive Bayes": GaussianNB(),
    "Logistic Regression": LogisticRegression(max_iter=1000)  # Increase
}

X_train_dense = X_train.toarray() if isinstance(X_train, np.ndarray) else
X_test_dense = X_test.toarray() if isinstance(X_test, np.ndarray) else X_

for name, classifier in classifiers.items():
    if name == "Naive Bayes":
        classifier.fit(X_train_dense, y_train)
    else:
        classifier.fit(X_train, y_train)

    if name == "Naive Bayes":
        y_pred = classifier.predict(X_test_dense)
    else:
        y_pred = classifier.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted')
    recall = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')

    print(f"{name} Evaluation Metrics:")
    print(f"Accuracy: {accuracy:.2f}")
    print(f"Precision: {precision:.2f}")
    print(f"Recall: {recall:.2f}")
    print(f"F1 Score: {f1:.2f}")
    print("-"*30)
```

```
SVM Evaluation Metrics:
Accuracy: 0.77
Precision: 0.77
Recall: 0.77
F1 Score: 0.77
------------------------------
Random Forest Evaluation Metrics:
Accuracy: 0.76
Precision: 0.76
Recall: 0.76
F1 Score: 0.75
------------------------------
KNN Evaluation Metrics:
Accuracy: 0.64
Precision: 0.65
Recall: 0.64
F1 Score: 0.64
------------------------------
Decision Tree Evaluation Metrics:
Accuracy: 0.69
Precision: 0.69
Recall: 0.69
F1 Score: 0.69
------------------------------
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[39], line 17
     15 for name, classifier in classifiers.items():
     16     if name == "Naive Bayes":
---> 17         classifier.fit(X_train_dense, y_train)
     18     else:
     19         classifier.fit(X_train, y_train)

File ~/anaconda3/lib/python3.11/site-packages/sklearn/naive_bayes.py:267, in GaussianNB.fit(self, X, y, sample_weight)
    265 self._validate_params()
    266 y = self._validate_data(y=y)
--> 267 return self._partial_fit(
    268     X, y, np.unique(y), _refit=True, sample_weight=sample_weight
    269 )

File ~/anaconda3/lib/python3.11/site-packages/sklearn/naive_bayes.py:428, in GaussianNB._partial_fit(self, X, y, classes, _refit, sample_weight)
    425     self.classes_ = None
    427 first_call = _check_partial_fit_first_call(self, classes)
--> 428 X, y = self._validate_data(X, y, reset=first_call)
    429 if sample_weight is not None:
    430     sample_weight = _check_sample_weight(sample_weight, X)

File ~/anaconda3/lib/python3.11/site-packages/sklearn/base.py:584, in BaseEstimator._validate_data(self, X, y, reset, validate_separately, **check_params)
    582         y = check_array(y, input_name="y", **check_y_params)
    583     else:
--> 584         X, y = check_X_y(X, y, **check_params)
    585     out = X, y
    587 if not no_val_X and check_params.get("ensure_2d", True):

File ~/anaconda3/lib/python3.11/site-packages/sklearn/utils/validation.py:1106, in check_X_y(X, y, accept_sparse, accept_large_sparse, dtype, order, copy, force_all_finite, ensure_2d, allow_nd, multi_output, ensure_min_samples, ensure_min_features, y_numeric, estimator)
   1101         estimator_name = _check_estimator_name(estimator)
   1102     raise ValueError(
   1103         f"{estimator_name} requires y to be passed, but the target y is None"
   1104     )
-> 1106 X = check_array(
   1107     X,
   1108     accept_sparse=accept_sparse,
   1109     accept_large_sparse=accept_large_sparse,
   1110     dtype=dtype,
   1111     order=order,
   1112     copy=copy,
   1113     force_all_finite=force_all_finite,
   1114     ensure_2d=ensure_2d,
   1115     allow_nd=allow_nd,
   1116     ensure_min_samples=ensure_min_samples,
   1117     ensure_min_features=ensure_min_features,
   1118     estimator=estimator,
   1119     input_name="X",
   1120 )
```

```
   1122 y = _check_y(y, multi_output=multi_output, y_numeric=y_numeric, es
timator=estimator)
   1124 check_consistent_length(X, y)

File ~/anaconda3/lib/python3.11/site-packages/sklearn/utils/validation.py:
845, in check_array(array, accept_sparse, accept_large_sparse, dtype, orde
r, copy, force_all_finite, ensure_2d, allow_nd, ensure_min_samples, ensure
_min_features, estimator, input_name)
    843 if sp.issparse(array):
    844     _ensure_no_complex_data(array)
--> 845     array = _ensure_sparse_format(
    846         array,
    847         accept_sparse=accept_sparse,
    848         dtype=dtype,
    849         copy=copy,
    850         force_all_finite=force_all_finite,
    851         accept_large_sparse=accept_large_sparse,
    852         estimator_name=estimator_name,
    853         input_name=input_name,
    854     )
    855 else:
    856     # If np.array(..) gives ComplexWarning, then we convert the wa
rning
    857     # to an error. This is needed because specifying a non complex
    858     # dtype to the function converts complex to real dtype,
    859     # thereby passing the test made in the lines following the sco
pe
    860     # of warnings context manager.
    861     with warnings.catch_warnings():

File ~/anaconda3/lib/python3.11/site-packages/sklearn/utils/validation.py:
522, in _ensure_sparse_format(spmatrix, accept_sparse, dtype, copy, force_
all_finite, accept_large_sparse, estimator_name, input_name)
    519 _check_large_sparse(spmatrix, accept_large_sparse)
    521 if accept_sparse is False:
--> 522     raise TypeError(
    523         "A sparse matrix was passed, but dense "
    524         "data is required. Use X.toarray() to "
    525         "convert to a dense numpy array."
    526     )
    527 elif isinstance(accept_sparse, (list, tuple)):
    528     if len(accept_sparse) == 0:

TypeError: A sparse matrix was passed, but dense data is required. Use X.t
oarray() to convert to a dense numpy array.
```

This code efficiently evaluates the effectiveness of machine learning classifiers by initialising a dictionary of them, training them on the dataset, predicting sentiment labels, and computing accuracy, precision, recall, and F1 score for each model. It then outputs these metrics (Géron, 2019). (42 words)

Bibliography

Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. O'Reilly Media.

Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media.

Grus, J. (2019). *Data Science from Scratch: First Principles with Python* (2nd ed.). O'Reilly Media.

McKinney, W. (2012). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O'Reilly Media.

Müller, A. C., & Guido, S. (2017). *Introduction to Machine Learning with Python: A Guide for Data Scientists*. O'Reilly Media.

Raschka, S., & Mirjalili, V. (2017). *Python Machine Learning* (2nd ed.). Packt Publishing.

VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.

Waskom, M. (2021). *Seaborn: Statistical Data Visualization*. Wiley.